

Interbloqueos

Módulo 7

Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"

Módulo 7: Interbloqueos

- Modelo de Sistema
- Caracterización de Interbloqueos
- Métodos para el Manejo de Interbloqueos
- Prevención de Interbloqueos
- Evasión de Interbloqueos
- Detección de Interbloqueos
- Recuperación de Interbloqueos

Objetivos

- Describir los interbloqueos, para asegurar que conjuntos de procesos concurrentes completen sus tareas.
- Presentar diferentes métodos para prevenir o evitar interbloqueos en un sistema de computación.

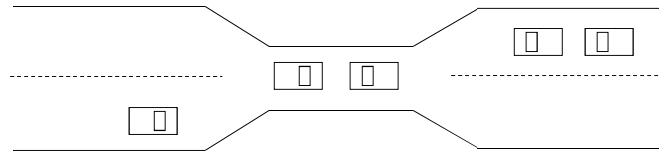
El Problema de Interbloqueo

- Un conjunto de procesos están en interbloqueo, si cada uno teniendo recursos, está esperando adquirir otro retenido por procesos en el conjunto.
- Ejemplo
 - Un sistema tiene 2 discos.
 - P_1 y P_2 cada uno tiene un disco y necesita otro.
- Ejemplo
 - semáforos A y B , inicializados a 1

P_0
wait (A)
wait (B)

P_1
wait(B)
wait(A)

Ejemplo del Cruce en el Puente



- Tránsito sólo en una dirección.
- Cada sección del puente puede verse como un recurso.
- Si ocurre un interbloqueo, puede ser resuelto si cada auto retrocede (se apropia de recursos y *rollback*).
- es posible que haya que retroceder varios autos.
- Es posible la inanición.

Modelo de Sistema

- Tipos de Recursos $R_1, R_2, \dots, R_m$
ciclos CPU, espacio de memoria, arch., dispositivos E/S
- Cada recurso tipo R_i tiene W_i instancias.
- Cada proceso utiliza un recurso como sigue:
 - requerimiento
 - uso
 - liberación

Caracterización de Interbloqueos

El Interbloqueo puede alcanzarse si se cumplen las cuatro condiciones simultaneamente.

- **Exclusión Mutua** : solo un proceso a la vez puede usar un recurso.
- **Retener y Esperar**: un proceso mantiene al menos un recurso y está esperando adquirir recursos adicionales retenidos por otros procesos.
- **No Apropiación**: Un recurso puede ser liberado solo voluntariamente por el proceso que lo tiene, después que haya completado su tarea.
- **Espera Circular**: existe un conjunto $\{P_0, P_1, \dots, P_n\}$ de procesos esperando, tal que P_0 está esperando por un recurso que es retenido por P_1 , P_1 está esperando por un recurso que es retenido por P_2 , ..., P_{n-1} está esperando por un recurso que es retenido por P_n , y P_n está esperando por un recurso que es retenido por P_0 .

Grafo de Alocación de Recursos

Un conjunto de vértices V y un conjunto de lados o aristas E (edges).

- V está particionado en dos tipos:
 - $P = \{P_1, P_2, \dots, P_n\}$, el conjunto formado por todos los procesos en el sistema .
 - $R = \{R_1, R_2, \dots, R_m\}$, el conjunto formado por todos los tipos de recursos del sistema.
- lado de requerimiento: es el lado dirigido $P_i \rightarrow R_j$
- lado de asignamiento: es el lado dirigido $R_j \rightarrow P_i$

Grafo de Alorcación de Recursos (Cont.)

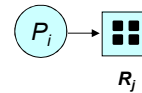
▪ Proceso



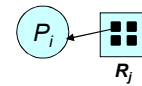
▪ Recurso tipo con 4 instancias



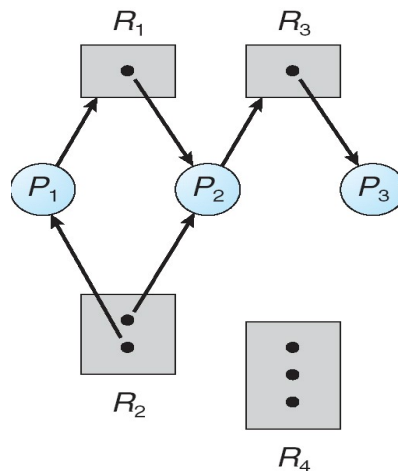
▪ P_i requiere una instancia de R_j



▪ P_i mantiene una instancia R_j

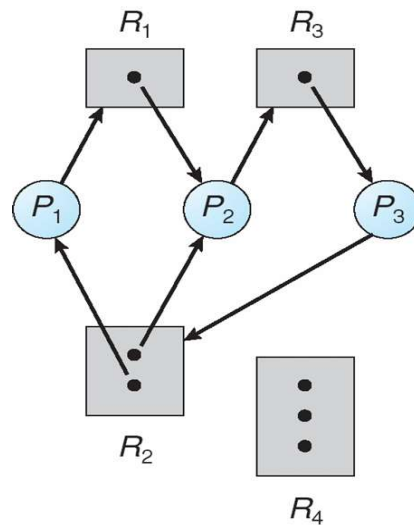


Ejemplo: Grafo de Alorcación de Recursos



Si tiene un ciclo, puede existir interbloqueo

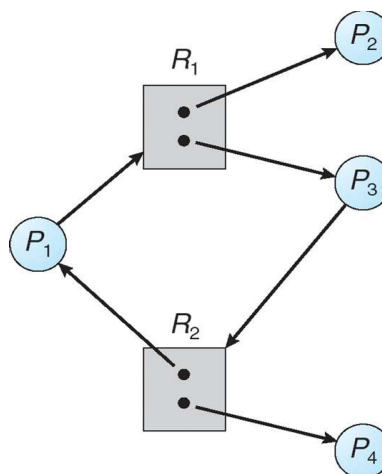
Grafo de Alorcación de Recursos con Interbloqueo



JRA © - LRS 2025

Sistemas Operativos – Interbloqueos

Grafo de Alorcación de Recursos con un Ciclo Sin interbloqueo



JRA © - LRS 2025

Sistemas Operativos – Interbloqueos

Cuestiones Básicas

- Si un grafo no contiene ciclos $\Rightarrow$ no hay interbloqueo.
- Si un grafo contiene un ciclo $\Rightarrow$
 - Si hay una sola instancia por tipo de recurso, entonces hay interbloqueo.
 - Si hay varias instancias por tipo de recurso, hay posibilidad de interbloqueo.

Métodos para Manejo de Interbloqueos

- **Impedir o evitar**, asegurando que el sistema no entrará *nunca* estado de interbloqueo.
- **Permitir** al sistema que entre en un estado de interbloqueo y luego *recuperarse*.
- **Ignorar el problema** y pretender que el interbloqueo nunca ocurrió en el sistema; usado en la mayoría de los sistemas operativos incluido UNIX.

Estrategias para Manejo de Interbloqueos

Para atacar el problema de interbloqueos se establecen en tres niveles de acción:

- ✓ antes que suceda,
- ✓ cuando los procesos están corriendo o
- ✓ con hechos consumados.

Estas estrategias son:

- Prevención
- Evasión
- Detección

Prevención de Interbloqueos

Restringir el modo en que se pueden realizar las solicitudes

- Exclusión Mutua – los recursos que pueden compartirse, no requieren esta condición, solo debe mantenerse para recursos no compartidos. *Por tanto no podemos negar esta condición.*
- Retención y Espera – debe garantizar que siempre que un proceso requiera un recurso, no lo esté reteniendo ningún otro proceso.
 - Un proceso debe requerir y alojar todos sus recursos antes que comience su ejecución, o
 - permitir a los procesos requerir recursos solo cuando no tenga ninguno.
 - Baja utilización de recursos; es posible inanición.

Prevención de Interbloqueos(Cont.)

- No Apropiación
 - Si un proceso que mantiene algunos recursos, requiere otro recurso y no le puede ser inmediatamente asignado, entonces todos los recursos mantenidos son liberados.
 - Los recursos desalojados son agregados a la lista de recursos por los cuales el proceso está esperando.
 - El proceso se reiniciará solo cuando pueda recuperar sus viejos recursos, tanto como los nuevos que está requiriendo.
- Espera Circular
 - impone un orden total de todos los tipos de recursos, y requiere que cada proceso solicite recursos en un orden creciente de enumeración.

Evasión de Interbloqueos

Requiere que el sistema tenga disponible alguna información adicional *a priori* .

- El modelo más simple y útil requiere que cada proceso declare el *número máximo* de recursos de cada tipo que puede necesitar.
- El algoritmo examina dinámicamente el estado de alocação de cada recurso, para asegurar que nunca se produzca una condición de espera circular.
- El estado de alocação de recursos está definido por:
 - el número de recursos disponibles y asignados, y
 - la demanda máxima de los procesos.

Estado Seguro

- Cuando un proceso requiere un recurso disponible, **el algoritmo debe decidir** si su asignación inmediata deja al sistema en un estado seguro.
- El sistema está en un **estado seguro** si existe una **secuencia segura** de ejecución de todos los procesos.
- La secuencia $\langle P_1, P_2, \dots, P_n \rangle$ es **segura** si por cada P_i , los recursos que P_i aún pueda requerir, sean satisfechos por recursos actualmente disponibles, más los recursos mantenidos por todos los P_j , con $j < i$.
 - Si los recursos que P_i necesita no están inmediatamente disponibles, entonces P_i puede esperar hasta que todos los P_j hayan finalizado.
 - Cuando P_j finaliza, P_i obtiene los recursos necesitados, ejecuta, retorna los recursos asignados, y termina.
 - Cuando P_i termina, P_{i+1} puede obtener los recursos que necesita, y así sucesivamente.

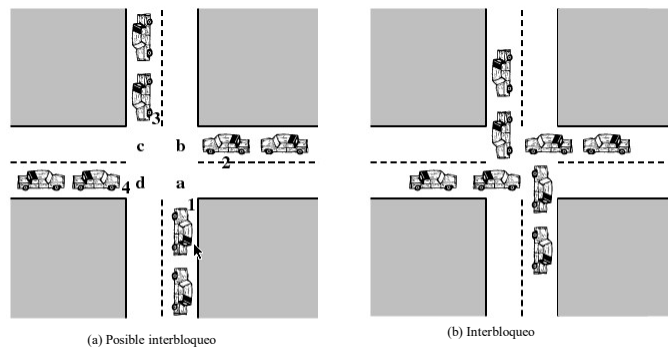
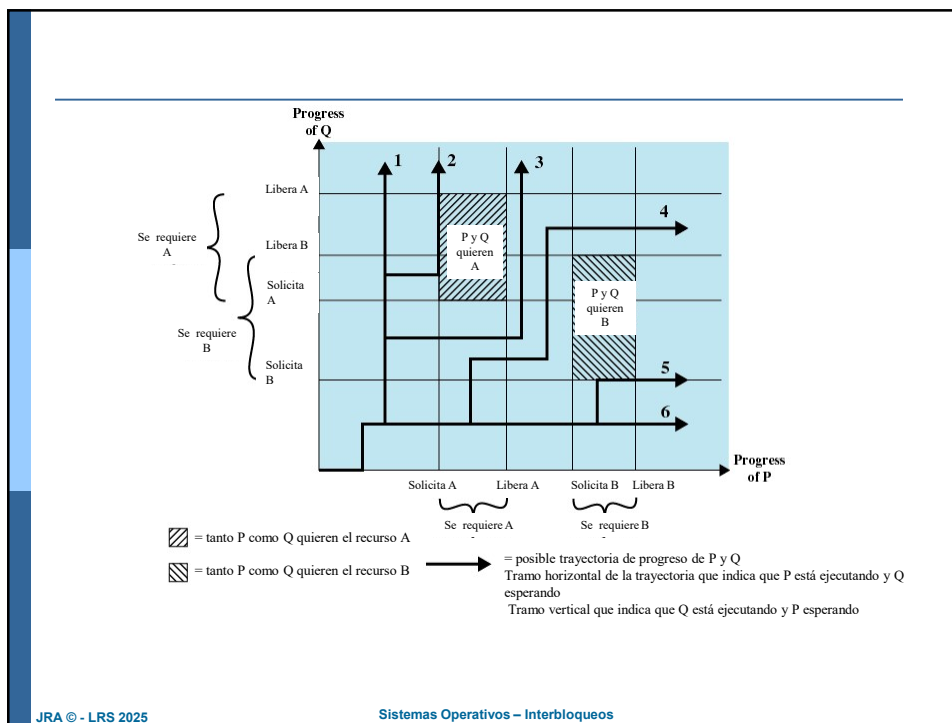
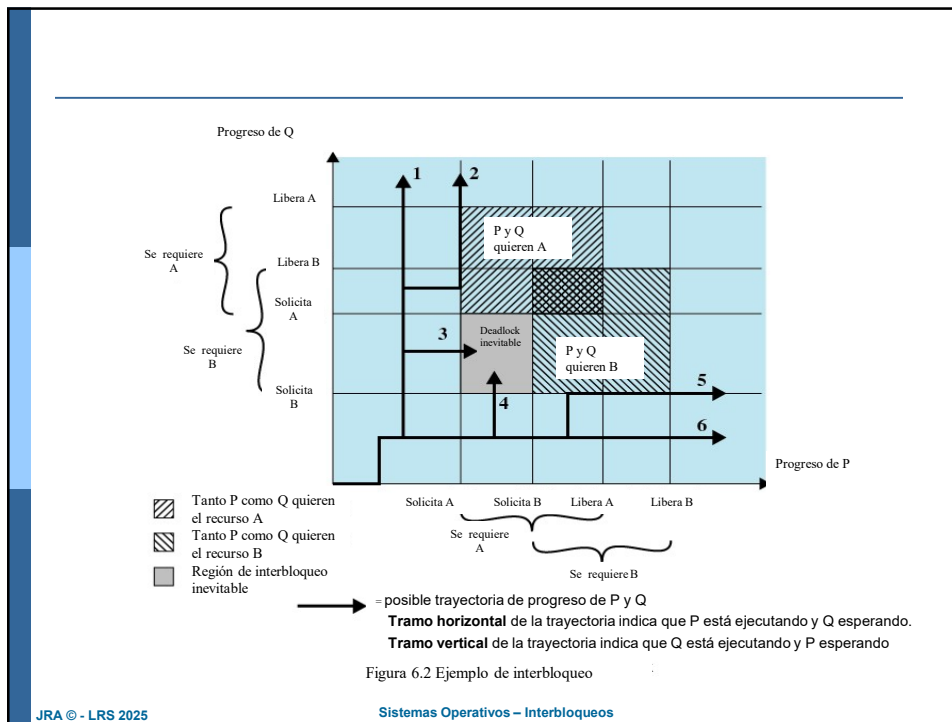


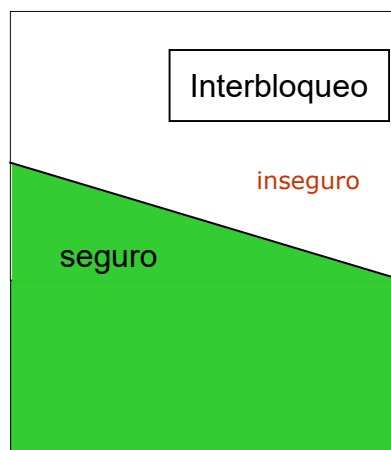
Ilustración del interbloqueo



Cuestiones Básicas

- Si un sistema está en un estado seguro $\Rightarrow$ no hay interbloqueo.
- Si un sistema está en un estado inseguro $\Rightarrow$ posibilidad de interbloqueo.
- Evasión $\Rightarrow$ asegura que el sistema *nunca* va a entrar en un estado inseguro.

Espacio de estados Seguro, Inseguro e Interbloqueo



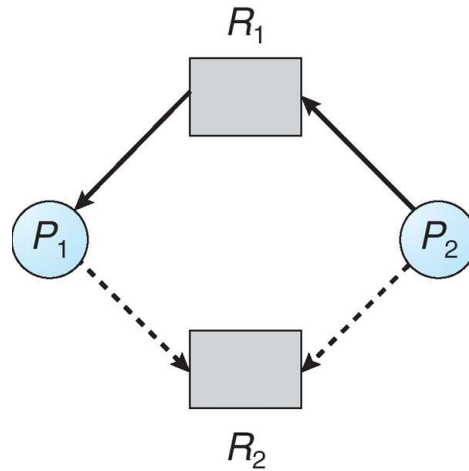
Algoritmos de Evasión

- Si sólo hay una instancia de un tipo de recurso.
Use un **grafo de asignación de recursos**.
- Para múltiples instancias de un tipo de recursos .
Use el **algoritmo del banquero**.

Algoritmo del Grafo de Asignación de Recursos

- **Lado de reclamo** $P_i \rightarrow R_j$ indica que el proceso P_i puede requerir el recurso R_j ; representado por una línea de puntos.
- El lado de reclamo se convierte en lado de requerimiento cuando el proceso **requiere** un recurso.
- Cuando un recurso es liberado por un proceso, el lado de asignamiento se reconvierte a *lado de reclamo*.
- Los recursos deben ser reclamados *a priori* en el sistema.

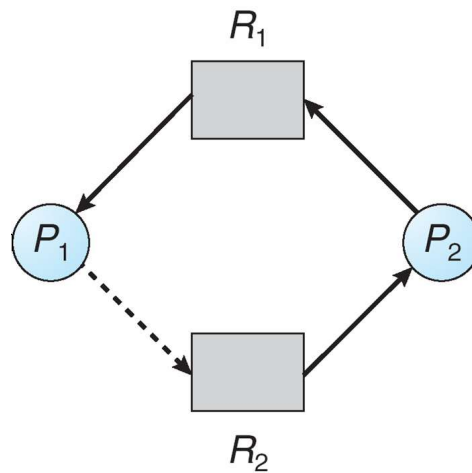
Grafo de Alorcación de Recursos para Evasi3n de Interbloqueos



JRA 3 - LRS 2025

Sistemas Operativos - Interbloqueos

Estado Inseguro en un Grafo de Alorcaci3n de Recursos



JRA 3 - LRS 2025

Sistemas Operativos - Interbloqueos

Algoritmo del Banquero

- Múltiples instancias.
- Cada proceso debe reclamar a priori el máximo uso.
- Cuando un proceso requiere un recurso puede tener que esperar.
- Cuando un proceso obtiene todos sus recursos debe retornarlos en una cantidad finita de tiempo.

Estructura de Datos del Algoritmo del Banquero

Sea n = número de procesos, y m = número de tipos de recursos.

- **Disponible**: Vector de longitud m . Si $\text{disponible}[j] = k$, hay k instancias del tipo de recurso R_j disponible.
- **Max**: matriz $n \times m$. Si $\text{Max}[i,j] = k$, entonces el proceso P_i puede requerir a lo sumo k instancias del recurso de tipo R_j .
- **Alocación**: matriz $n \times m$. Si $\text{Alocación}[i,j] = k$ entonces P_i tiene alocadas k instancias de R_j .
- **Necesidad**: matriz $n \times m$. Si $\text{Necesidad}[i,j] = k$, entonces P_i puede necesitar k instancias mas de R_j para completar su tarea.

$$\text{Necesidad}[i,j] = \text{Max}[i,j] - \text{Alocación}[i,j].$$

Algoritmo de Solicitud de Recursos para el Proceso P_i

$Request_i$ = vector de requerimiento para el proceso P_i .

$Request_i[j] = k$ P_i quiere k instancias del tipo de recurso R_j .

1. Si $Request_i \leq Necesidad_i$ vaya al paso 2. Sino **condición de error, el proceso ha excedido su máximo.**
2. Si $Request_i \leq Disponible$, vaya al paso 3. Sino P_i debe esperar, los recursos no están disponibles.
3. Se **simula** asignar los recursos requeridos por P_i modificando el estado como sigue:

$Disponible := Disponible - Request_i$

$Alocación_i := Alocación_i + Request_i$

$Necesidad_i := Necesidad_i - Request_i$

- Si **seguro** $\Rightarrow$ los recursos son asignados a P_i .
- Si **inseguro** $\Rightarrow P_i$ debe esperar, y es restaurado el viejo estado de asignación de recursos

Ejemplo del Algoritmo del Banquero

- 5 procesos P_0 a P_4 ; 3 recursos tipo A (10 instancias), B (5 instancias), y C (7 instancias).
- Instantánea en el instante T_0 :

	<u>Alocación</u>			<u>Max</u>			<u>Disponible</u>			<u>Necesidad</u>		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2	7	4	3
P_1	2	0	0	3	2	2				1	2	2
P_2	3	0	2	9	0	2				6	0	0
P_3	2	1	1	2	2	2				0	1	1
P_4	0	0	2	4	3	3				4	3	1

- El contenido de la matriz Necesidad es definido como:

$Max - Alocación$

El sistema está en un estado seguro dado que la secuencia $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisface los criterios de seguridad.

Example (Cont.): P_1 requiere (1,0,2)

- Verifique que Request $\leq$ Disponible (esto es, (1,0,2) $\leq$ (3,3,2) $\Rightarrow$ verdadero.

	<u>Alocación</u>	<u>Necesidad</u>	<u>Disponible</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Ejecutando el algoritmo muestra que la secuencia $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisface el requisito de seguridad.
- ¿Puede el requerimiento (3,3,0) por P_4 ser satisfecho?
- ¿Puede el requerimiento (0,2,0) por P_0 ser satisfecho?

Detección de Interbloqueos

- Permite al sistema entrar en estado de interbloqueo

El sistema debe proporcionar:

- Un algoritmo de Detección
- Un esquema de Recuperación

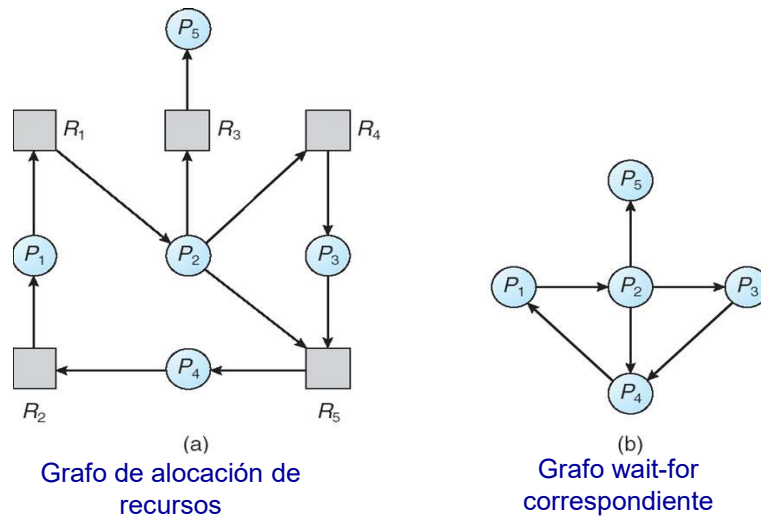
Instancia Simple de Cada Tipo de Recurso

- Se mantiene un grafo *wait-for* (grafo de espera)
 - Los nodos son procesos.
 - $P_i \rightarrow P_j$ si P_i está esperando por P_j .
- Periódicamente se invoca un algoritmo que busca ciclos en el grafo.
- Un algoritmo para detectar un ciclo en un grafo requiere un orden de n^2 operaciones, donde n es el número de vértices en el grafo.

JRA © - LRS 2025

Sistemas Operativos – Interbloqueos

Grafo de Alocación de Recursos y Grafo Wait-for



JRA © - LRS 2025

Sistemas Operativos – Interbloqueos

Varías Instancias de un Tipo de Recurso

- *Disponible*: Vector de longitud m indica el número de recursos disponibles de cada tipo.
- *Alocación*: Una matriz de $n \times m$ que define el número recursos de cada tipo alocados actualmente a cada proceso.
- *Request*: Una matriz de $n \times m$ que indica el requerimiento actual efectuado por cada proceso. Si $Request[i,j] = k$, entonces el proceso P_i está requiriendo k instancias más del recurso de tipo R_j .

Ejemplo de Algoritmo de Detección

- Cinco procesos P_0 a P_4 ; tres tipos de recursos A (7 instancias), B (2 instancias), y C (6 instancias).
- En el instante T_0 :

	<u>Alocación</u>			<u>Request</u>			<u>Disponible</u>		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

- La secuencia $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ es segura.

Ejemplo (Cont.)

- P_2 requiere una instancia adicional del tipo C.

<u>Request</u>			
	A	B	C
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- ¿Estado del sistema?
 - Puede reclamar recursos mantenidos por el proceso P_0 , pero son insuficientes para completar otros procesos;
 - Existe interbloqueo consistente de los procesos P_1 , P_2 , P_3 y P_4 .

Uso del Algoritmo de Detección

- Cuándo debemos invocar el algoritmo de detección depende de:
 - ¿Con qué *frecuencia* es probable que ocurrirá un interbloqueo?
 - ¿*Cuántos* procesos serán afectados cuando ocurra un interbloqueo?
- Si el algoritmo de detección es invocado arbitrariamente, puede haber muchos ciclos en el grafo de recursos y no se puede decir cual de los muchos procesos interbloqueados **causó** el interbloqueo.

Recuperación: Terminación de Procesos

- Aborto de todos los procesos interbloqueados.
- Aborto de un proceso a la vez hasta que el ciclo de interbloqueo haya sido eliminado.
- ¿En qué orden se elige abortar?
 - Prioridad del proceso.
 - Cuánto tiempo ha computado el proceso, y cuánto falta para completar.
 - Recursos que el proceso ha usado.
 - Recursos que el proceso necesita para completar.
 - Cuántos procesos necesitarán ser terminados.
 - ¿Es el proceso interactivo o batch?

JRA © - LRS 2025

Sistemas Operativos – Interbloqueos

Recuperación: Apropiación de Recursos

- Selección de una víctima – minimiza costos.
- Rollback – retorna a algún estado seguro, reinicia el proceso desde ese estado.
- Inanición – algunos procesos pueden ser elegidos siempre como víctimas, incluir un número de rollbacks en el factor de costo.

JRA © - LRS 2025

Sistemas Operativos – Interbloqueos

Fin

Módulo 7

Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"